Next assignment: transactions

Decades: databases systems have handled concurrency/parallelism with <u>transactions</u>

```
begin;
  update some value
  update some other value
  read some values
end;
```

all happens atomically ↓ or not at all

want to prevent someone else from seeing partially finished results

Parallel prog community: can we do the same w/ general programming?

- experimental libraries for doing this
  - Python
  - Rust
  - C++

Assignment: implat this for Kotlin

Specifics.
We can't modify Kotlin itself.
Instead, all transaction reads/writes will be on specicl we create to do it.

```
fun main() {                          ✓ we are creating this class
    var x = TxObject<Int>(0)
    var y = TxObject<Double>(3.2)
    atomic {                          ← a function we make
        x.write(3)
        y.write(x.read() + 2)
    }
}
```

In Kotlin, f is a function

```
f { a b }   ⟷   f({ a b })
```

① STM.kt → where we define atomic

↑

"software transactional memory"

② TxObject.kt → where the TxObject code does .read .write

③ TxManager.kt

- keep track of whether or not a xaction is active or not
- keeps bookkeeping info for a xaction

Interlude on nulls in Kotlin

The Java Null Pointer Exception (NPE) is reviled and hated by many

- major source of challenges

Many more modern langs are introducing null-safe types of various sorts

- if something shouldn't be null, don't allow it to be
- if it should, don't compile the code if the code doesn't properly check for null before using the object.

In Kotlin, regular types (Strings, Int, etc) can't hold nulls.

?, e.g. String? indicates it can hold nulls.

!! - I'm going to use this pointer anyway, even if null. I take on the risk of an NPE

How do we make transactions work?

Each thread at the start of a xaction makes a list of all of the reads and writes that have been made.

```
atomic {
    x.write(3);
    x.write(7);
}
```

Keep a list of all the old and new values as they happened

list

x 0 ⇒ 3

x 3 ⇒ 7

Keep a private history of all the changes you made

At the time of commit, you lock all the variables you need (if you can), and then you execute the changes, making sure that the starting value hasn't

changed.

If any badness happens:
- can't get locks
- initial values changed since you started
- ...?

Abort transaction, and try again starting over from scratch.

TxManager: start() will create a map to connect a TxObject with the changes that were made.

TxObject	changes
x	3 → 5
y	"hello" → "bye"

Every thread needs its own TxManager object, since it tracks all of the details for an active xaction on that thread.

- You'll need a "thread local" variable.